TP05: Real-Time Data Streaming and Processing Using Apache Kafka

Question 1. What problem does Kafka solve?

Apache Kafka solves the problem of transferring and processing large amounts of data in real time. It allows different applications to exchange data reliably and efficiently through a distributed streaming platform.

Question 2. Define Producer, Consumer, Broker, Topic, Partition, Offset, and Consumer Group.

- **Producer:** An application that sends messages to Kafka.
- **Consumer:** An application that receives and processes messages from Kafka.
- **Broker:** A Kafka server responsible for storing and managing messages.
- **Topic:** A named channel used to organize messages.
- **Partition:** A subdivision of a topic that allows data to be distributed and processed in parallel.
- **Offset:** The position of a message within a partition.
- **Consumer Group:** A group of consumers that cooperate to read messages from a topic.

Question 3. What is the difference between a Topic and a Partition?

A topic is a logical category where messages are stored, while a partition is a subdivision of that topic. Partitions allow Kafka to distribute data and increase parallel processing.

Question 4. Why do we use `auto_offset_reset='earliest'`? When does it apply?

This setting allows a consumer to start reading from the earliest available message when it does not have a valid committed offset. If a valid offset already exists, the consumer continues from that position.

Question 5. What is the purpose of the Group ID? What happens if we use a new Group ID?

The Group ID identifies a consumer group. Consumers with the same Group ID share the work of reading messages. If we use a new Group ID, Kafka treats it as a new consumer group, which has its own offsets and can read the messages independently.

Question 6. Why do we use JSON? What is one limitation of JSON?

JSON is used because it is a simple and lightweight format for exchanging structured data between applications. One limitation is that it does not support all complex data types directly and requires serialization and deserialization.

Question 7. How does Docker Compose simplify Kafka deployment?

Docker Compose simplifies deployment by defining the Kafka service, its configuration, ports, and volumes in one YAML file. It allows us to start and manage the Kafka container using simple commands.

Question 8. What are the limitations of a single-broker setup compared to a production cluster?

A single-broker setup has limited fault tolerance and availability. If the broker fails, Kafka becomes unavailable. It also does not provide real replication across multiple brokers. Production clusters use several brokers to improve reliability, scalability, and fault tolerance.

Question 9. What evidence can demonstrate the successful execution of the TP?

The following screenshots can demonstrate the successful execution:

- Docker container status showing that the Kafka broker is running.
- Kafka topic description showing the topic and its partitions.
- Producer output showing that messages were sent successfully.
- Consumer output showing received sensor data and temperature alerts.